Tutorial 25 — LLM-Driven Evolutionary Search

Chapter 10 with two AlphaEvolve circle-packing problems

ENGR 689 Tutorial Team

November 2, 2026

Contents

1 Agentic evolutionary frameworks **2**

2 The circle-packing problem **2**
2.1 Circle packing: the problem . . . 2
2.2 Circle packing: the goal . . . 3

3 How we run and compare packing programs **4**
3.1 How a packing program searches . . . 4
3.2 What is a trial? . . . 5

4 10.1 The evolutionary loop **6**
4.1 Our strategy: evolve packing programs . . . 6
4.2 Five choices define the search . . . 7

5 10.2 Mutation operators **7**
5.1 Classical variation . . . 7
5.2 A language model writes the mutation . . . 8
5.3 Context joins the two course spines . . . 9

6 10.3 Selection and population management **9**
6.1 Elitism protects measured progress . . . 9
6.2 Selection pressure and population collapse . . . 10

7 10.4 Fitness design **10**
7.1 Packing score and program fitness . . . 10

8 10.5 Diversity and exploration **12**
8.1 Preserve different search behaviors . . . 12
8.2 Exploration, exploitation and cost . . . 12

9 Two published circle-packing problems **13**
9.1 Demo 1: follow three mutations . . . 13
9.2 Read the first run's evidence . . . 14
9.3 Demo 2: the 32-circle problem . . . 15

9.4 Demo 2: inspect adaptation and limits 15
9.5 Name the framework we observed 16

A Appendix: architecture and data flow **16**
A.1 One server coordinates the experiment 16
A.2 Request, context and source response 17
A.3 Execution, checking and feedback 17

References **18**

1 Agentic evolutionary frameworks

First we will explain the circle-packing problem and what makes one arrangement better than another. Then we will use the evolutionary framework in Chapter 10 to improve programs that search for these arrangements. The two demonstrations use 26 and 32 circles. These are the published mathematical tasks from AlphaEvolve. Our local controller and short Luna runs are classroom implementations, not a reproduction of the unpublished AlphaEvolve engine or its reported records.

2 The circle-packing problem

2.1 Circle packing: the problem

What are we trying to arrange?
Place **26 or 32 circles** inside a square with side length 1. Choose each circle's **center** (its location) and **radius** (half its width). The radii may differ.

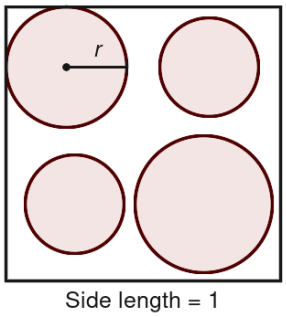


Figure 1: Four circles illustrate the rules.

Every circle must stay inside the square. Circles may touch each other and the boundary, but they must **not overlap**.
A valid packing satisfies these rules for every circle. The number of circles stays fixed within each problem.

Begin with the geometry before introducing an algorithm. A center is the point in the middle of a circle. A radius is the distance from that center to its edge, so the diameter is twice the radius. A unit square has side length one. In each problem the number of circles is fixed: 26 for Demo 1 and 32 for Demo 2. Their sizes can differ. We choose both their centers and their radii.

The picture shows four circles so the rules are easy to see. It is an illustration, not the 26- or 32-circle demo output. The dot marks one center and the segment labeled r runs from that center to the circle's edge. Its plotted center/radius triples are (0.22,0.22,0.22), (0.74,0.22,0.18), (0.25,0.72,0.18), and (0.72,0.72,0.25). All circles fit inside the square and their interiors are disjoint.

A packing is valid only if every circle lies entirely inside the square and no two circle interiors overlap. Touching is allowed. For a center $c_i = (x_i, y_i)$ and radius r_i, these rules are $r_i \geq 0$, $r_i \leq x_i \leq 1 - r_i$, $r_i \leq y_i \leq 1 - r_i$, and $\|c_i - c_j\|_2 \geq r_i + r_j$ for each distinct pair. Zero radii are allowed under the demonstration's contract and remain visible in its reports. The next frame introduces the score, separately from these validity rules.

The two cases use the mathematical formulation in AlphaEvolve Appendix B.12 (AlphaEvolve team 2025). The local initial methods, controller, Luna model and compute budget are our classroom implementation. No published answer coordinates are hidden in the initial arrangements.

2.2 Circle packing: the goal

Which valid packing is better?
Packing score: $S = r_1 + \cdots + r_N$, the sum of all radii. **Larger is better.**

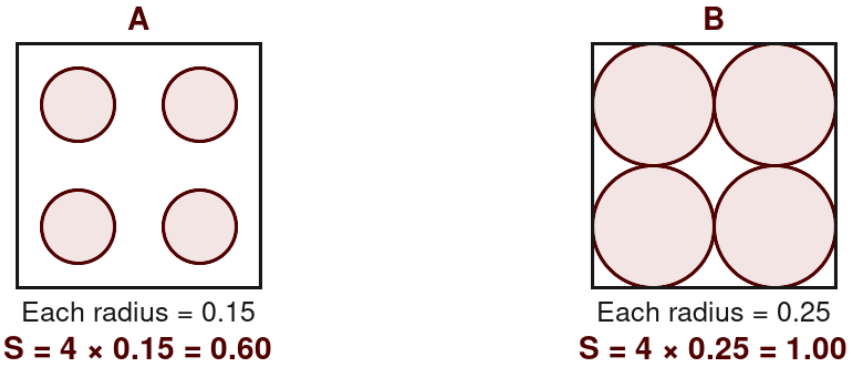


Figure 2: Same four centers, different radii. Both packings are valid.

B has the larger score. Circle positions limit how large the radii can be. We need a strategy to search for better packings.

Ask students to add the radii before revealing the conclusion. Point to the visible space around A's circles and then to the touching circles in B. Four times 0.15 is 0.60. Four times 0.25 is 1.00. Both drawings use centers (0.25,0.25), (0.75,0.25), (0.25,0.75) and (0.75,0.75). Arrangement B touches the boundary and neighboring circles, which the rules allow. These exact-coordinate drawings are teaching illustrations, not measured results from either demonstration or a claim about the best possible four-circle packing.

The score is the sum of radii. Covered area would use times the sum of squared radii, which is a different objective. We also keep the number of circles fixed when comparing scores. A raw sum for 32 circles cannot establish improvement over a run with 26 circles.

This simple example improves by increasing radii at fixed centers. In a crowded arrangement, further changes can require moving centers: making room for one circle can reduce the room available for others. Finding a good arrangement therefore involves interacting choices. We will use

an evolutionary framework to improve the Python programs that search for packings. A program is the method; a packing is the arrangement it produces. We check and score the arrangement to evaluate the method. This strategy can find better constructions without certifying a global optimum.

3 How we run and compare packing programs

3.1 How a packing program searches

What does a program do to improve one packing?
Example: **Greedy Local Center Search**. Start with circle centers. **Radius allocation** chooses radii for those fixed centers to maximize their sum while satisfying the packing rules.
Propose a random change to one center. The **move scale** controls the typical size of that change. Recalculate radii and score. The **move acceptance rule** keeps the proposed change only if the score increases. This cycle is one **search step**.
Repeat the steps, then return the best packing found. This loop changes a packing. Later, we will change the program itself.

Explain the numerical method before discussing edits to its code. This frame describes the disclosed Greedy Local Center Search program. It starts with the supplied centers. The helper radii(centers) solves a linear program for the radii: the centers and their pairwise distances are fixed, the radii are the unknowns, and every wall and pair imposes a linear bound. The helper maximizes the sum of radii under those bounds with the common numerical safety margin. Choosing good center positions remains the search problem.

One search step copies the centers, randomly chooses one circle and proposes random changes to its x and y coordinates. The move scale is the standard deviation of each coordinate change before clipping centers to their allowed coordinate range. It controls typical displacement, not a fixed step length or a maximum distance. The implemented scale decreases during a run: scale $= 0.04 \times (1 -$ k/steps$) + 0.002$. Here k is the step index. The coefficient 0.04 controls the larger early movements and the added 0.002 keeps a small positive scale. Changing that coefficient to 0.02 reduces typical movement at the same step; it does not make every displacement exactly 0.02.

The program reallocates radii after the proposed center move and computes the new sum. Its move acceptance rule is a condition in an if statement. If the current sum is 2.00 and the proposed sum is 2.03, the greedy program keeps the proposed centers. If the proposed sum is 1.98, it keeps the previous centers. These numbers are teaching examples. The separate Simulated Annealing of Centers program can accept a temporary score decrease with a controlled probability and retains its best packing separately. That is a different move acceptance rule, not permission to accept overlapping geometry.

The program repeats numerical steps until it returns or its execution is stopped by a limit. We call one execution of the whole program a trial in the next frame. Keep the levels distinct: a center move changes an arrangement inside a program; a later mutation changes the program's source; evolutionary selection compares whole programs after evaluation.

3.2 What is a trial?

What do Trial 1, Trial 2 and Trial 3 mean?

A **trial** is one execution of an unchanged program from given centers. It may contain many search steps. Limits: **80 requested steps**, **8 seconds maximum**.

A **random seed** is a number that initializes the program's random choices. Our defaults are:

Trial	1	2	3
Random seed	3	4	5

Use the same three starting packings and seeds for every program. This shared set of starting packings is the **packing bank**.

A trial is one call to solve(initial, seed, steps), executed in an isolated Python worker. The source stays unchanged during that execution. It receives a copy of one specified set of starting centers, one numerical random seed, and the requested step count. The program can make many proposed center moves before returning one final arrangement. A checked intermediate arrangement is progress within the trial, not another completed trial.

The controller evaluates a program three times. With the default first seed of 3, Trial 1 uses seed 3, Trial 2 uses seed 4, and Trial 3 uses seed 5. These numbers are ordinary defaults, not special mathematical conditions. A random seed initializes a pseudo-random number generator, making its generated sequence repeatable for the same program and inputs. A changed program may consume random numbers differently, so equal seeds do not force two programs to make identical moves. Hard time limits and numerical execution conditions can also affect reproducibility. Numerical seeds do not control Luna's wording.

The three trials use three specified starting packings. Every competing program receives copies of the same bank, with the corresponding seed and limits. Eight seconds is a hard wall-time cap including worker startup; 80 is a requested number of search steps. A constructor may finish with fewer steps, and generated source may use the work request differently. Equal limits do not imply equal numerical effort. A timeout is an incomplete trial, not a successfully completed eight-second score.

In the default refinement protocol, the bank for the next round comes from the best previous packing for each seed. Freeze those centers, re-evaluate parents and the new program from copies of that bank, and compare only their current-bank scores. The fixed-start option instead keeps the original bank. The next frame explains fitness: all three trials must complete and be valid, then their middle score ranks the program. One trial is a numerical execution, a model call proposes source code, and an evolutionary round coordinates proposal, evaluation and selection. They are different units of work.

4 10.1 The evolutionary loop

4.1 Our strategy: evolve packing programs

How can we improve the method that finds the packings?
Run programs to produce packings. **Fitness:** the median (middle) score of three complete, valid trials. Keep useful programs as **parents**.

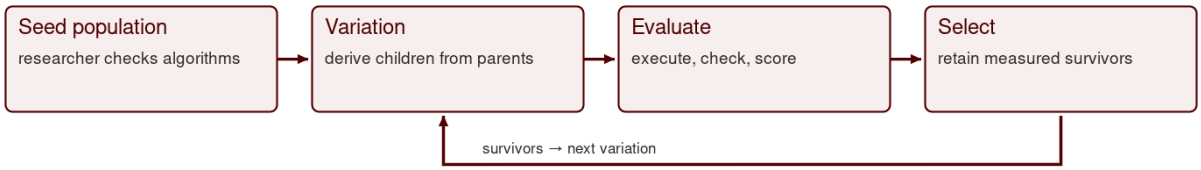


Figure 3: Initialize once; evaluate, select and mutate repeatedly.

Luna, our language model, edits parent code. Test each new program before keeping it.

We now have a problem, a numerical search method and a three-trial comparison protocol. Our proposed strategy improves the programs used to search for packings. The group of programs is the population. Programs chosen for further changes are parents. Luna is the language model used to write a modified program, called a child. That source change is mutation. Running the child's three trials and checking their results tells us whether to keep the program, which is selection. Fitness means the measured performance used in that decision: here it is the middle of three complete, valid packing scores. The later fitness frame works through the calculation and the chapter's equivalent negative-sign convention.

Point through the diagram in order. Initialization happens once. Later rounds use selected programs and their results to propose changes. This is the Chapter 10.1 evolutionary loop: the next attempt builds on survivors rather than starting as an unrelated guess. The outside controller stores programs and measurements. Luna's model weights stay fixed, while the information sent in its next request changes. The checker supplies evidence beyond the model's explanation of its proposal.

A survivor is a program. Executing it produces a circle arrangement. Keeping a successful arrangement can also provide a numerical starting point for later programs, but that is separate from inheriting program source. Our default demonstration exposes both: program ancestry and a frozen bank of the previous best packings.

4.2 Five choices define the search

What must we define before running this search?
$(\mathcal{A}, f, \mu, \sigma, P_0)$: allowed programs, measured performance (**fitness**), how programs change (**mutation**), which programs we keep (**selection**), and the starting group (**initial population**).

$$P_{t+1} = \sigma(P_t \cup \{\mu(a) : a \in \sigma(P_t)\}) .$$

Choose parents → mutate → pool parents and children → select survivors.
New programs compete with the parents. A worse new program does not have to replace a useful parent.

This is Chapter 10, Definition 10.2 and Equation 10.1. The course minimizes fitness f. For our maximization problem, use negative median verified sum of radii. The artifact space consists of Python packing programs with a fixed function interface and bounded execution. Mutation is a Luna request containing actual selected parent source. Selection retains at most three complete programs. Checkboxes choose the initial population; they do not restrict the source changes Luna may propose.

The displayed recurrence is the chapter's conceptual definition. Our controller sends up to three selected parents in one request and asks for one child, rather than making one request for every parent. It then evaluates that child and selects from parents plus child. This is a declared classroom choice, not a hidden implementation of every feature of AlphaEvolve. The chapter's fixed fitness f corresponds directly to our fixed-start control. Default iterative refinement uses conditional fitness f(a | B_t), where the incoming packing bank is explicit evaluation state. Re-evaluating parents makes each round comparable; it does not make medians from different banks stationary program quality.

5 10.2 Mutation operators

5.1 Classical variation

How can we change the packing program's code?
Point mutation: edit one constant, such as 0.04 → 0.02 for smaller center moves. **Subtree replacement:** replace the condition that decides whether to keep a proposed center move.
Crossover: combine code from two parents, such as one program's starting construction with another's move acceptance rule.
These edits change the program, not just one packing. Evaluate the resulting program with the same three-trial protocol before selecting it.

Recall the earlier numerical-search frame before introducing these operators. It showed one unchanged program proposing center moves, allocating radii, and applying a move acceptance rule. We now edit the code defining those operations. Chapter 10.2.1 introduces point mutation, subtree mutation and crossover using expression trees. An expression tree represents an expression as nested operations and values. A subtree is one complete nested part, such as the condition in an if statement.

A point mutation can change the coefficient controlling random center movements from 0.04 to 0.02. The actual program also decreases its scale with the step index, so this is an edit to the schedule's coefficient, not a promise that every move has length 0.02. A subtree replacement

can replace the strict-improvement condition with another compatible Boolean condition. A different acceptance rule might permit occasional score decreases while preserving valid geometry. Crossover can combine compatible source components from two parents. These examples translate the classical vocabulary to packing programs; the controller does not implement a hidden classical genetic-programming engine.

Such edits are hypotheses about a better method. Smaller moves may refine an arrangement but explore less widely. Accepting some temporary losses may help exploration but can also waste work. Run each changed program using the same three starting conditions and limits as its competitors, then compare complete valid trial scores. Inner move acceptance decides which center positions a program keeps; outer evolutionary selection decides which programs survive. The next frame introduces Luna as the source-code mutator used in the demonstration.

5.2 A language model writes the mutation

What can a mutator change when it reads the parent code and its results?

1. Retain the parent's center search and radius allocator.
2. Change center-move size, which centers move, or the move acceptance rule.
3. Luna returns edited **Python source** and a hypothesis.
4. An isolated interpreter executes it; an independent checker evaluates the returned circles.

Luna implements the proposed code change; Python executes it. A reasoned edit becomes evidence only after evaluation.

Chapter 10.2.2 replaces blind variation with knowledge-guided variation. For example, a parent may perturb one center at a time and stop improving. The mutator can hypothesize that coordinated moves or a restart schedule will escape that basin. It writes the new function; the local Python interpreter does not translate the prose hypothesis into an algorithm.

Our response contains round_id, parent_ids, hypothesis and source. The source defines solve(initial, seed, steps). NumPy, math, a fixed-center radius allocator and a progress reporter are supplied. A source diff exposes what was retained and changed. The geometry checker and survivor rule are outside the editable source. Bubblewrap isolates generated code from credentials, private repository files and the host network. The model has no execution tools during its proposal call.

AlphaEvolve evolves code and evaluates it externally (AlphaEvolve team 2025). We use the published circle-packing tasks with our own small controller and TAMU Luna. The public results repository does not release the original discovery engine. Do not describe our initial numerical methods as the algorithms AlphaEvolve discovered.

5.3 Context joins the two course spines

Which evidence should the next mutation see?
Parent source **and scores**; successful and failed attempts; task constraints and useful domain knowledge; population similarity and remaining budget.
Evaluation becomes fitness. Context engineering drives mutation. The next request carries the previous round's actual results.
External memory makes the next proposal informed. A plausible explanation does not replace a measured trial.

These are the four context ingredients in Chapter 10.2.3 and the convergence of the two spines in Figure 10.2. Our domain knowledge includes the nonoverlap constraints and the fact that radius allocation is a linear program once centers are fixed. Source, scores, failures and tried IDs expose what the search already knows. The context labels the current packing bank, so scores from different starting conditions are not silently mixed.

The server's request panel shows the actual provider payload; the response panel shows the public stream. Authorization is added separately inside the model worker. Hidden reasoning is neither requested as teaching evidence nor displayed. Numerical seeds control starting packings and numerical moves. They do not determine Luna's wording. The optional structure-only control withholds scores and history, but a rigorous context comparison must additionally freeze identical parent programs and histories across the two conditions.

6 10.3 Selection and population management

6.1 Elitism protects measured progress

If a mutation is worse, must the current champion disappear?
Pool parents with the new child. Retain the best complete programs under the declared fitness rule.
For a maximized quality, elitism makes the observed best stay level or improve **under comparable evaluation conditions**.
A failed mutation need not erase a useful parent. Preserve the best valid packing separately from program fitness.

Chapter 10.3.1 describes carrying elite candidates forward. The lower-is-better course fitness is monotonically non-increasing when the same deterministic measured fitness is used. Equivalently, maximized observed quality is non-decreasing. This is a property of retaining a measured incumbent, not a theorem of global convergence or future quality on new instances.

Our default uses inherited packing banks. At each round, freeze the same incoming packings for the parents and the child, re-evaluate the parents, and select within that bank. An old program median from a different bank is not directly comparable. The cumulative best valid construction is a separate monotone curve because its verified geometry is retained. A valid progress packing can survive in that archive even when a program later times out and receives no eligible fitness.

6.2 Selection pressure and population collapse

Why can keeping only the highest scores make search stagnate?
Top-k: rank everyone and retain the best k. **Tournament:** sample a group and retain its best; smaller tournaments can preserve more alternatives.
Collapse: several survivors become small rewrites of the same search. Current fitness does not reveal every parent's future descendants.
Strong selection exploits evidence but can discard useful routes. Our top-three rule does not guarantee diversity.

Chapter 10.3.2 compares top-k and tournament selection; 10.3.3 identifies population collapse. Our implemented rule is deterministic: all three trials must complete and pass geometry checks; then rank by median sum of radii, minimum sum, and stable source identity. A child below the champion may still survive if it belongs among the three best programs. Do not teach elitism as a rule that rejects every nonchampion.

Programs with different IDs may have nearly identical behavior. Normalizing syntax prevents comments alone from creating another genome, but it is not a semantic diversity measure. Several schedules differing only in a constant can still explore the same region. The guide names our three initial mechanisms so students can inspect their differences before deciding whether the population is varied enough. Tournament selection is explained here but is not implemented as a hidden classroom feature.

7 10.4 Fitness design

7.1 Packing score and program fitness

How do we turn packing scores into a program's fitness?
Packing score: $S = \sum_i r_i$, after checking the geometry. A scorer that ignores overlap can reward invalid packings.
Program fitness: run three complete, valid trials with the same starting packings and numerical limits for every program.

$$F = \text{median}(S_1, S_2, S_3).$$

Example scores 1.90, 2.10, 2.00 give $F = 2.00$.
The demo maximizes F. Chapter 10 minimizes $f = -F$. Any invalid or incomplete trial makes the program ineligible.

Chapter 10.4 treats the evaluator as the fitness function and warns that persistent optimization finds weaknesses in it. Here we explicitly compare a flawed scorer with the corrected evaluator, rather than merely presenting an invalid output. The flawed scorer returns the sum of radii and omits pair checks. It ranks two circles of radius 0.30 at centers (0.30, 0.50) and (0.70, 0.50) above two radius-0.25 circles at (0.25, 0.50) and (0.75, 0.50). The claimed gain is 0.10, but the first pair overlaps by 0.20. An optimizer rewarded by that flawed rule has an incentive to enlarge overlapping circles. The correct geometry checker rejects the apparent winner. These are instructor-made, numerically checked toy examples, not Luna outputs. The algorithm guide contains a revealable scorer illustration.

A production witness must contain exactly the required number of finite circle triples. Radii must be nonnegative, circles must fit inside the square, and every pair must avoid overlap. The server

independently recomputes both geometry and summed radii; it ignores a program's claimed score. Numerical tolerance is 1e-9 in unit-square units. A common radius-allocation construction margin of 1e-8 is applied before a final witness is returned and scored. No invalid output is silently repaired during evaluation.

Separate the two levels explicitly. One execution returns a packing. After independently checking its geometry, add its radii to obtain that trial's score S_j. A program receives three trials, using the same starting packings, trial seeds and numerical limits as its competitors. Its primary ranking value is $F = \text{median}(S_1, S_2, S_3)$, which the demonstration maximizes and displays as a positive median. Sort the illustrative scores 1.90, 2.10 and 2.00 into 1.90, 2.00 and 2.10. The middle score is 2.00, so $F = 2.00$. These numbers illustrate the calculation and are not measured demo results.

Chapter 10 uses a minimization convention. Its equivalent fitness is $f(a \mid B_t) = -F(a \mid B_t) = -\,\text{median}_{j=1,2,3}\, S(a; z_{t,j})$, where a is a program and B_t contains the three starting packings $z_{t,j}$ frozen for this round. Thus the example has $f = -2.00$. A larger F corresponds exactly to a smaller f. The server ranks by the positive median, then by the minimum trial score, then by stable program identity. It does not need to store the negative value to implement this ordering.

A failed trial is not quietly dropped when calculating fitness. Any invalid or incomplete trial makes the whole program ineligible for selection. A valid intermediate packing remains a separate useful construction. Comparisons use one frozen bank; medians from different banks do not measure program quality under the same conditions. This distinction is essential for anytime numerical search.

Feasibility checking does not establish algorithm transfer. Freeze program source before evaluating fresh numerical starts, exclude those validation results from mutation prompts and survivor decisions, and report them separately. The original runs used seeds 3, 4 and 5. A separate numerical-only check uses previously unused seeds 101, 102 and 103, common fixed starts, the same requested work and the same hard trial limit. The selection rule was fixed before seeing validation scores: compare the best initial program with the highest-ranked surviving new Luna program. The guide shows the actual results and source identities. This is a small post-search check, not a broad generalization study or a second live evolution.

Returning a fixed valid construction can be a legitimate solution to a fixed packing problem; a hard-coded layout alone is not proof of reward gaming. The loophole here is accepting infeasible geometry. Overfitting claims concern performance outside the observed starts or problem instances and require a separate evaluation contract. Multiobjective quality and cost are developed in the exploration frame and guide. The changing center positions define a nonconvex landscape; the fixed-center radius linear program does not solve that whole problem.

8 10.5 Diversity and exploration

8.1 Preserve different search behaviors

How can useful alternatives survive beside the current champion?
Islands: evolve separate populations, with occasional migration. **MAP-Elites:** retain the best program in each behavior bin, such as one-center versus coordinated moves.
Novelty search: favor behaviors unlike those already seen. Different source text alone does not establish different search behavior.
Preserve several ways to search. These are possible extensions; our top-three controller does not implement a diversity archive.

Chapter 10.5.1 motivates preserving several search histories. An island owns a population and mutation history. Occasional migration shares programs; overly frequent migration can make the islands alike, while too little sharing can leave each stalled. Three evaluation seeds are not three islands, and the two circle counts are different problems rather than populations exchanging discoveries.

The Multi-dimensional Archive of Phenotypic Elites (MAP-Elites) organizes candidates by chosen behavior descriptors and keeps a high-quality representative in each bin (Mouret and Clune 2015). For a packing-program archive, illustrative descriptors could measure the typical number of centers moved per proposal and the frequency of restarts. A one-center local search and a coordinated-move search could occupy different bins even if one currently has a lower verified sum. Their quality would still need comparable evaluation conditions. Those descriptors would require instrumentation; the present worker does not record them as a ready-made archive.

Novelty search instead scores behavioral difference from previously observed behaviors (Lehman and Stanley 2011). A possible descriptor distance could use normalized numerical search statistics. It is not a measure of how unusual a program's variable names sound, and novelty alone does not certify packing quality. These are alternative selection designs whose performance depends on the task and descriptors.

Our controller retains the top three complete programs and rejects exact syntax duplicates. It has no islands, behavioral bins or novelty score. The guide works through the conceptual archive example so students can recognize both its purpose and its absence from the bounded demo.

8.2 Exploration, exploitation and cost

When should we refine, explore—or stop spending?
Exploit: refine. **Explore:** change methods. **Stop:** respect the cap.
Hypothetical policies, same test conditions:

Policy	Median sum	Numerical seconds
A	2.40	1
B	2.41	5

Illustrative utility $S - 0.01t$ **favors A:** $2.39 > 2.36$**.** Our demo reports costs separately.

Chapter 10.5.2 frames the explore–exploit tension. Exploitation refines useful inherited structure; exploration tests a different search mechanism. Internal numerical exploration, such as accepting temporary losses, is different from an outer mutation changing the program. Neither a large source edit nor a fluent hypothesis guarantees an improvement.

The table is invented to explain a decision, not measured Luna performance. Both policies must first be valid and evaluated comparably. B offers 0.01 more quality but takes four more numerical seconds. Neither dominates: A is faster and B has higher quality. A quality–time Pareto view therefore keeps both as options. If we explicitly choose a value of 0.01 quality units per second, maximizing utility U=S−0.01t gives A=2.39 and B=2.36. The price is a teaching assumption, not a measured API tariff or universal preference. A smaller price could reverse the decision.

Chapter 10.8 expresses the corresponding minimization form as cost-adjusted fitness. Cost may mean marginal API dollars or a declared resource measure, with a coefficient that converts units. Generation cost belongs to the path that produced a program; it is not an intrinsic property of source code. Avoid double-counting shared preparation and distinguish per-program evaluation time from all model calls and numerical effort spent finding it. Source length is not automatically a good proxy for either runtime or algorithm quality.

The implemented demo keeps quality primary and reports numerical effort and model usage separately. It does not apply the illustrative price, a complexity penalty or a Pareto archive. Three model calls, 35 seconds per call, eight seconds per trial and a 240-second run deadline bound work. An optional predeclared plateau stopping rule could save further requests; it is not the current automatic policy. The current run can continue its planned rounds after a plateau so students can inspect the mechanism.

No number of calls is universally optimal. Three calls per demonstration are a teaching budget: the first child, a proposal informed by its evaluation, and another chance to inspect adaptation or plateau. This budget is far smaller than the research-scale searches in the AlphaEvolve report. A flat or failed run remains useful evidence. Do not add artificial motion or repeated requests to force a desired trajectory.

9 Two published circle-packing problems

9.1 Demo 1: follow three mutations

Which part of the parent survives, and does the changed program earn its place?
Evaluate the initial population → inspect parent source and context → send Luna round 1 → read the actual code diff and verified packing.
Repeat for rounds 2 and 3. Inspect inherited packing-bank IDs, complete trial scores, failures and survivor decisions.
Follow the real chain: source → execution → checked geometry → fitness → selection → next context.

Open the integrated demo from this frame. Select LIVE LLM, 26 circles, three calls, mutation, evidence-rich context and inherited packings. Check the selected initial programs and evaluate them. This starts numerical work only; every model call waits at an explicit gate.

Show the three starting programs in the guide: Perturbed Grid Constructor, Greedy Local Center Search and Simulated Annealing of Centers. Each uses the same procedure to choose radii once the centers are fixed. The grid constructor intentionally spends less effort. Compare actual numerical cost as well as quality. All programs in a round receive copies of the same starting packings for seeds 3, 4 and 5.

At the first gate, point to the incoming packing bank: it contains the best valid construction found for each seed. Parents are re-evaluated from that bank before their scores enter the context. When

sending a round, identify the actual provider request, public response and source string. Show who implemented the proposal: Luna wrote the Python; the isolated interpreter executes that exact admitted source. A live intermediate packing is independently verified, but it is not yet complete program fitness.

After evaluation, compare all three trials and the actual top-three decision. Before sending the next call, show that the latest results have entered the context and the packing bank has been updated. Keep failures or plateaus visible. Use the remaining teaching time for questions and inspection if computation finishes early; no artificial delay is inserted. A genuine recording may be used when needed, with a persistent RECORDED REPLAY label.

9.2 Read the first run's evidence

What improved: one packing, the program's measured fitness, or both?
A witness shows one feasible packing. After evolution, compare the frozen initial and final programs on the original starting bank. Three complete trials supply each median.
Keep construction progress and program comparison separate. Report the actual outcome, calls and numerical effort.

After the last round finishes, scroll to "Did the programs improve from the original starts?" and press "Compare initial and final programs on original starts." This runs numerical trials only, with no Luna calls. Initial programs and final survivors receive the original three center arrays, the same seeds and the same limits. Initial programs and final survivors appear in separate tables. A source shared by both groups is tested once and appears in both tables with the same scores. Read each table by median and minimum; failures have no eligible median. The comparison has its own 240-second cap and Stop comparison control. It does not alter the evolutionary population or feed results back to Luna. Original starts were used during evolution, so this is an original-start comparison, not held-out validation. If time is short, start this comparison while explaining the evidence and let it finish before interpreting the scores.

Ask students to point to the record supporting each claim. The circle view includes program identity, seed, validity and summed radii. The table includes complete or failed trials, medians and current-bank selection. The cumulative construction curve retains valid witnesses; it must not be mislabeled as a fixed-protocol program-fitness curve when banks change.

Do not substitute expected improvements for measured outcomes. If an eligible child is worse than the champion, it can still enter another slot. If a trial fails, the program has no eligible median. Matching a historical literature score would still not prove optimality. A three-call classroom run cannot establish broad model or algorithm superiority. The guide's separate numerical validation illustrates how to test frozen programs beyond the evolution starts, rather than treating geometric feasibility as proof of transfer.

9.3 Demo 2: the 32-circle problem

Does the same program-evolution mechanism still work when the number of circles changes?
Use $N = 32$, the second unit-square case in AlphaEvolve Appendix B.12. Keep the objective, checker, trial seeds, source interface and numerical limits declared.
Change the problem instance, then inspect adaptation. A higher raw sum for 32 circles is not evidence of a better algorithm than the 26-circle run.

Reset and select 32 circles. The problem fingerprint changes, as do the number of wall and pair constraints. The method signatures accept n=len(initial), so a program must not assume 26 rows. This is a second published mathematical problem, not an arbitrary graph substitution. Its score scale is different; do not compare raw sums between the two circle counts as though they measured the same instance.

Use the same three disclosed initial methods and seeds so students can recognize the mechanism, but do not call this a controlled test of generalization. The programs are evolved with access to each task. The point is to observe the same Chapter 10 components operating on a changed instance. A separate withheld-size evaluation would be required for a transfer claim. A separate fixed-start check of already-frozen programs is now available in the algorithm guide. It used new seeds 101–103, zero additional model calls and no feedback into evolution. The initial baseline and a surviving new program were selected before inspecting those results. This small check concerns fresh starts at the same N; it is not an unseen-size transfer study.

9.4 Demo 2: inspect adaptation and limits

Does Luna change its search after seeing the 32-circle trial evidence?
Evaluate 32-circle parents → send three gated requests → inspect actual source edits, verified packings and the feedback entering each next request.
Keep invalid proposals, incomplete executions and repeated ideas visible. Stop cancels owned work; replay remains explicitly recorded.
The mechanism is testable even when improvement is absent. Evidence decides which proposed programs survive.

Follow the same procedure as demo one. Make the changed n visible in the request context and in the circle count of returned witnesses. A response that hard-codes the wrong number is invalid; the interpreter does not silently fix it. A source syntax error, duplicate, invalid packing or hard timeout remains part of the outcome and does not trigger an unannounced free retry.

The class should see the proposal arrive, the function execute and the resulting geometry checked. Compare the child with parents evaluated from the same incoming packing bank. Inspect the actual next request rather than asserting that the model learned. Its weights remain unchanged; its context contains the search memory. If a recorded run is used, show its source identity and recorded timing, while making clear that no new request is being made.

9.5 Name the framework we observed

Can you identify every Chapter 10 component in the live record?
$\mathcal{A}$: Python packing programs. f: verified current-bank trial fitness. μ: Luna's code mutation. σ: complete-program selection. P_0: checked initial programs.
Context proposes; external evaluation measures; selection preserves. For your research problem, define these five choices before spending search budget.

Return to the tuple after both demos so its components are now concrete. Students should distinguish the evolving program, the numerical packing returned by execution, and fitness used to select programs. They should also be able to explain ancestry, inherited numerical starts, context feedback and evaluator independence.

Chapter 10 supplies the conceptual structure. AlphaEvolve supplies the two published problem formulations and a real precedent for evolving search programs. Our bounded implementation supplies inspectable classroom evidence. The mathematical task matches the paper; the original engine, model ensemble, compute scale and reported optimized constructions are not reproduced. Those limits matter when explaining what the demonstration establishes.

A Appendix: architecture and data flow

A.1 One server coordinates the experiment

Who owns the population, execution and measurements?

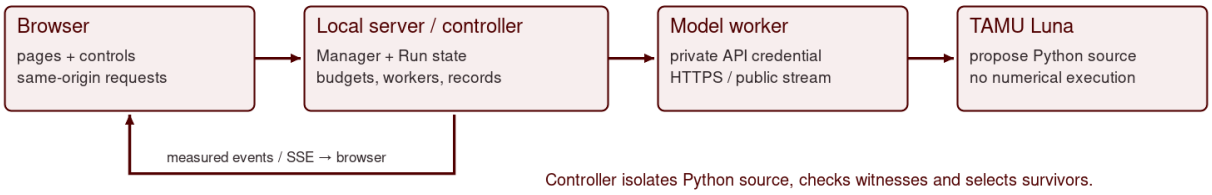


Figure 4: The server coordinates isolated execution and a separate provider worker.

The controller owns the loop. Only the model worker contacts TAMU; a separate isolated worker executes generated Python.

The same loopback server serves the presentation, demo, algorithm guide, exchange, notes and PDFs. Browser controls create a run, start numerical evaluations and explicitly release each model gate. Server-sent events expose actual progress and support reconnect without repeating work. The model worker alone receives the configured provider credential. The generated program runs inside Bubblewrap with read-only runtime code, no host network and no repository, run-log or private-home mounts. Geometry is checked again in the server outside that code.

A.2 Request, context and source response

What crosses the model API boundary?
Send the task, parent source, current-bank trial results, history and limits. Authorization is a separate HTTPS header.
Return JSON: `round_id`, `parent_ids`, `hypothesis`, `source`. The source defines `solve(initial, seed, steps)`.
A source string is executable code after admission. The hypothesis is an explanation, never the authoritative score.

The TAMU transport uses protected.gpt-5.6-luna, public streaming and no tools. It sends system instructions plus a JSON response schema and serialized context. The actual body is displayed on the demo page; the key is never part of it. Source identity is normalized syntax plus executor version. An inherited proposal must name actual surviving parent IDs. An independent proposal must use an empty parent list. The controller rejects duplicates and malformed responses without silent retries.

A.3 Execution, checking and feedback

When can a returned program become a survivor?
Validate source → isolated trials → verify every circle and pair → aggregate three complete scores → select within the current packing bank.
Keep exact request/response records, valid witnesses and failures. The next context carries the measured history.
Execution turns a proposal into evidence. Live LLM, numerical-only and recorded replay remain distinct throughout.

Each numerical trial has an eight-second hard wall cap, including its isolated startup. Each model request has a 35-second cap, and the run has a 240-second deadline with bounded owned-process cleanup. Stop and Reset cancel the owned worker tree, not unrelated processes. Missing isolation or dependencies blocks execution instead of falling back to unsafe or fabricated work.

The numerical-only mode evaluates disclosed initial source programs without contacting Luna. Live mode performs real requests only at explicit gates. Replay is registered only from genuine provider records with matching task, source identities, geometry and transcript provenance. It reproduces recorded events, and reveals responses only as its timeline reaches them. Replay is not a new measured result under a changed executor.

References

AlphaEvolve team. 2025. "AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery." Google DeepMind. 2025. https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf.

Lehman, Joel, and Kenneth O. Stanley. 2011. "Abandoning Objectives: Evolution Through the Search for Novelty Alone." *Evolutionary Computation* 19 (2): 189–223. https://doi.org/10.1162/EVCO_a_00025.

Mouret, Jean-Baptiste, and Jeff Clune. 2015. "Illuminating Search Spaces by Mapping Elites." *arXiv Preprint arXiv:1504.04909*. https://arxiv.org/abs/1504.04909.